Generating Fiducial Labels for MedalCare-XL with ECGdeli

Methodology, Quality Control, and Integration into the Delineation Pipeline

1. Purpose and context

MedalCare-XL is the intended training corpus for the ECG-delineation model. The dataset is distributed as signals only with the 12-lead waveforms and the electrophysiological parameter files used to generate them, but no fiducial (P/QRS/T onset-peak-offset) annotations.

In the interim, we generate fiducial labels with ECGdeli, the open-source delineation toolbox from the same group (KIT-IBT) that produced MedalCare-XL, and the same tool the dataset authors used to extract interval features for their own technical validation (Table 6 of the published paper's data descriptor).

2. The data and the manifest

The official MedalCare-XL dataset release is described as containing approximately 16,900 ECGs. The version processed in this report yielded 16,848 ten-second 12-lead ECGs at 500 Hz, across a healthy class and seven pathologies (myocardial infarction is the bulk, with six occlusion/transmurality sub-classes), with valid, uniquely resolved signal and metadata paths. Each ECG is stored leads-by-samples (12 rows × 5000 columns, no header) in three versions: raw (noise-free), filtered, and noise-added.

Disease class	ECGs	Note
sinus	1,300	healthy control
mi	7,806	6 MI sub-classes (LAD/LCX/RCA × transmurality)
avblock	1,300	1st-degree AV block
lbbb	1,300	left bundle branch block
rbbb	1,298	right bundle branch block
lae	1,300	left atrial enlargement
fam	1,300	fibrotic atrial cardiomyopathy
iab	1,244	interatrial conduction block

2.1 Manifest (preserving the class information)

To avoid losing the morphology/disease information when the signals are flattened, we first built a single index **medalcare_manifest.csv**, with one row per ECG, containing record_id, disease_class, mi_subclass, split, batch, run_id,

sampling rate, and the paths to all three signal versions and the parameter files. All 16,848 records resolve to unique IDs and verified paths. This manifest drives every downstream step, so the 84,000 files never have to be handled by hand.

3. Delineation with ECGdeli

ECGdeli operates on an in-memory, matrix-oriented samples × leads format, whereas MedalCare stores leads × samples. Rather than duplicate 27 GB, the driver transposes each file on load. The processing per record is -> read CSV → transpose to 5000×12 → (optional isoline correction) → Annotate_ECG_Multi (signal, 500, 'PQRST') → map the fiducial-point table (FPT) columns into the project schema → append one row per beat-per-lead. The driver (**run_ecgdeli_medalcare.m**) is resumable and was validated on a 20-record smoke test before the full run.

3.1 Fiducial-point table (FPT) column mapping

ECGdeli returns P/QRS/T onsets, peaks, and offsets, where the individual Q, R, and S peaks are measured natively (so Q and S are measured, not derived). Column 6 = R-peak is confirmed in **Annotate_ECG_Multi.m** while columns 1/3/4/12 are confirmed in the MedalCare synthesis code.

FPT col	Fiducial	FPT col	Fiducial
1	P onset	7	S peak
2	P peak	8	QRS offset
3	P offset	10	T onset
4	QRS onset	11	T peak
5	Q peak	12	T offset
6	R peak	—	(U wave: not detected)

Output Convention - sample indices are 0-based; milliseconds = sample × 1000 / 500. Disease class and, for infarction, the MI subclass are carried on every row: mi_subclass holds the detailed label (LAD/LCX/RCA at each severity, with the LCX anterior/posterior variant) and mi_group holds the normalised six-category group, so morphology information is never lost.

How Q and S are measured - "measured" here means the driver reads the Q, R and S positions straight from ECGdeli's fiducial-point table (columns 5, 6 and 7) as produced by the toolbox's own QRS-delineation routine so for each beat it simply takes fpt(b,5), fpt(b,6) and fpt(b,7) with no computation of its own.

The Q, R and S positions are taken directly from ECGdeli's QRS-delineation output rather than being derived by the present pipeline. Their accuracy nevertheless remains dependent on ECGdeli and may be reduced in low-amplitude, fractionated or atypical QRS morphologies.

3.2 Smoke test and full run

The 20-record smoke test (avblock) produced a physiologically and clinically consistent result, confirming the input orientation, schema mapping, unit conversion, output generation and physiological plausibility before scaling.

Biomarker (lead II median)	Value	Interpretation
PQ/clinical PR interval	214 ms	prolonged — consistent with 1st-degree AV block (the avblock class)
QRS duration	132 ms	matches MedalCare's simulated QRS width (paper Table 6)
QT interval	386 ms	within range
Heart rate	77 bpm	physiological

Schema, ms = sample×2, and presence flags all verified with zero errors. The full run then labelled all 16,848 records, producing 2,646,204 beat-lead fiducial rows in **medalcare_fiducials_ecgdeli.csv.**

From 16,848 records to 2,646,204 rows - the count grows because the schema stores one row per beat per lead, not one per recording. Each ten-second ECG contains on the order of 8-14 heartbeats, and ECGdeli delineates all twelve leads independently, so a single record expands to roughly (beats × 12) rows. Across the dataset this is about 16,848 records × 12 leads × ~13 beats ≈ 2.6 million, the exact figure of 2,646,204 is the sum of the actual beats detected in every lead of every record, an average of about 13 beats per lead.

4. Quality control of the pseudo-labels

4.1 QC design (qc_review_list.py)

A streaming Python screen (it processes the 2.65 M-row file without loading it into memory) classifies every beat into three tiers. A small tolerance ignores trivial boundary wobbles, so the review list stays targeted.

What informs the design, and what an inversion is - The automated QC screen targets two readily testable classes of annotation failure: violations of expected temporal ordering and derived intervals outside predefined

plausibility ranges. It does not detect every possible localization error, particularly consistently shifted but physiologically plausible boundaries.

An inversion here means a pair of adjacent fiducials that appear out of order, meaning a fiducial that should come later is assigned an earlier sample index than the one before it, for example a P offset placed before the P peak, or a T onset placed after the T peak. It is determined by walking the eleven fiducials of each beat in their canonical order (P onset, P peak, P offset, QRS onset, Q, R, S, QRS offset, T onset, T peak, T offset) and comparing every neighbouring pair; whenever the earlier point's sample index exceeds the later point's, that pair is flagged and the size of the gap, in samples, is recorded. That size sets the tier, an inversion of at most three samples (≤ 6 ms) is negligible boundary wobble and left clean, whereas a structural inversion inside the QRS block, a gross inversion beyond twenty samples (≈ 40 ms), or an out-of-range interval is treated as critical.

- **CRITICAL - review or exclude - a structural QRS-order violation, a QRS/QT/PR interval outside a predefined plausibility range, a gross P/T boundary inversion (> 20 samples ≈ 40 ms), or a missing R peak.**
- **MINOR - a moderate P/T boundary inversion (3–20 samples), treated as lower priority and left out of the review list.**
- **CLEAN - no tested issue, or an inversion of ≤ 3 samples (≤ 6 ms, negligible).**

Two outputs are written -> **medalcare_qc_review_list.csv** (one row per CRITICAL beat, with its failure reasons and key fiducial samples) and **medalcare_qc_record_summary.csv** (every record ranked worst-first by critical rate). The full code is reproduced in Appendix A.

4.2 QC results

Tier	Beats	Share
CRITICAL (review)	104,171	3.94 %
MINOR (small P/T wobble)	477,300	18.04 %
CLEAN	2,064,733	78.03 %

Structural QRS-ordering violations were rare, occurring in 50 beat–lead rows. This indicates that the returned QRS fiducials are generally internally ordered but does not by itself establish their localization accuracy. The critical flags are dominated instead by interval and T-boundary problems - QT out-of-range (52,202), gross P/T boundary inversion (48,977), PQ/clinical PR out-of-range (40,416) and QRS duration (11,350). These reason counts are not mutually exclusive, because one beat can contribute to more than one category.

Per beat vs per record-lead - The rates above count individual beats, which overstates how much of the dataset needs work. Each MedalCare-XL ECG is synthesised from common underlying P-wave and QRST templates, so the beats of a (record, lead) share the same source morphology but are not exact duplicates: RR timing varies and the QRS-offset–T-offset segment is stretched or compressed. A lone flagged beat among clean siblings may therefore represent a transient delineation glitch rather than a systematically broken record. Counting at the (record, lead) level, 2.10% of units are persistently flagged under the operational threshold of ≥50% of the unit's beats flagged. Approximately 54% of the flagged beats sit in units below this threshold — these are lower-priority for manual review and may be suppressed by an aligned per-record consensus based on intervals or R-aligned relative positions. The recurring patterns collapse to 4,251 persistently flagged units, concentrated in MI/LBBB/RBBB, across 2,300 recordings. Manual review can therefore be targeted to these units rather than all 104,171 flagged beat–lead rows. One representative beat can guide the correction of a unit, but corrected boundaries must be transferred to the remaining beats using R-peak alignment and, where necessary, temporal scaling. The per-beat 3.94% is thus an upper bound on individually flagged rows, while the persistently flagged units provide a more realistic review set; the 50% threshold does not prove that all remaining units are error-free.

Disease class	Persistent-flag rate (record-lead)	Persistent / total units
lbbb	4.52%	705 / 15,600
mi	2.73%	2,558 / 93,672
rbbb	2.04%	317 / 15,576
iab	1.51%	225 / 14,928
fam	0.97%	152 / 15,600
sinus	0.96%	149 / 15,600
lae	0.84%	131 / 15,600
avblock	0.09%	14 / 15,600

4.3 Distribution of QC flags by disease class

The per-class critical-flag rate follows the expected pattern whereby the ventricular-pathology classes (LBBB, MI, RBBB) have the highest rates, while the healthy and atrial-disease classes have the lowest. This gives empirical, quantified evidence of where the pseudo-labels need the most scrutiny and the strongest case for pursuing the simulation-derived ground truth for those classes, but the QC flag rate does not by itself measure true annotation error.

Disease class	Critical rate	Critical / total beats

lbbb	7.8 %	16,217 / 208,428
mi	4.3 %	51,726 / 1,204,620
rbbb	4.2 %	8,276 / 198,816
iab	3.2 %	6,325 / 199,392
sinus	3.1 %	6,590 / 210,480
lae	2.9 %	6,032 / 209,076
fam	2.7 %	5,768 / 210,072
avblock	1.6 %	3,237 / 205,320

4.4 Population-level validation against the paper's Table 6

We extracted the same six timing biomarkers from our ECGdeli labels on the healthy sinus cohort, across roughly 200,000 beat–lead observations over all twelve leads, and compared the per-lead means against the simulated values in Table 6 of the published paper. Given that that reference was also produced with ECGdeli, the consistency check here assesses whether we drive the tool similarly to the authors, as we don't have an independent ground-truth validation. Table 6 is the only per-lead numeric reference the descriptor provides: the preprint gives only the lead-II Figure 5 and the per-class Figure 6, with no equivalent table, so the per-lead comparison below is necessarily against the published paper's Table 6. As Section 4.6 shows, QT differs systematically between Table 6 and the preprint, so the QT row documents a reference discrepancy rather than independently validating either value.

Lead	P dur	QRS dur	T dur	PQ int	QT int	RR int
I	117 / 124	131 / 131	171 / 178	119 / 128	379 / 311	759 / 758
II	125 / 128	126 / 126	182 / 182	124 / 127	386 / 317	759 / 758
III	156 / 165	126 / 127	182 / 183	160 / 172	379 / 307	759 / 758
aVR	118 / 127	128 / 129	178 / 179	116 / 126	387 / 319	759 / 758
aVL	138 / 154	128 / 129	180 / 183	145 / 169	373 / 299	759 / 758
aVF	136 / 141	125 / 125	183 / 184	137 / 143	381 / 310	759 / 758
V1	136 / 141	129 / 129	177 / 181	154 / 161	374 / 304	759 / 758
V2	153 / 156	136 / 136	175 / 176	178 / 182	360 / 288	759 / 758
V3	150 / 154	132 / 133	179 / 179	170 / 174	359 / 286	759 / 758
V4	137 / 141	127 / 127	179 / 180	144 / 149	363 / 290	759 / 758
V5	125 / 129	124 / 124	176 / 177	122 / 127	380 / 311	759 / 758
V6	120 / 123	126 / 127	174 / 175	115 / 119	387 / 321	759 / 758

Each cell shows our mean vs the published Table 6 "sim" mean, in milliseconds (QT highlighted). The QT reference column is the published Table 6 value; the preprint provides no per-lead QT table, but its Figure 5 (lead II, ~385 ms) is closer to our QT.

The agreement is close for five of the six biomarkers. QRS duration is essentially identical to the published values (mean absolute difference 0.3 ms across all twelve leads), and RR interval (0.8 ms), T duration (1.8 ms), PQ interval (7.5 ms) and P duration (6.1 ms) all reproduce the reference within single-digit-to-low-tens of milliseconds with the

larger P-wave gaps falling, as expected, in the low-amplitude P leads (III, aVL). This supports reproducibility of our ECGdeli processing and the dataset's published population statistics, but not absolute fiducial accuracy.

The sixth biomarker, QT, reads +70.6 ms against the published Table 6 (a uniform 66–74 ms across leads). Our QT — about 386 ms in lead II — is close to the value shown in the descriptor's own preprint (Figure 5, ~385 ms), whereas the published Table 6 reports ~317 ms. The available evidence therefore indicates a systematic difference in processing, aggregation, software version, or T-offset convention between the references. Agreement in PQ/clinical PR and QRS duration supports reproducibility of those interval calculations but does not prove that QRS onset or T offset is exact. The QT discrepancy is retained as an unresolved reference difference rather than attributed definitively to either Table 6 or our labels.

4.5 Full reproduction of the population statistics

To turn the Table 6 check into a thorough reproducibility test, we recomputed every population statistic the descriptor reports: the healthy timing means and their standard deviations, the five amplitude features, and the per-class distributions of the descriptor's Figure 6. Timing statistics were computed from the complete set of 2,646,204 beat–lead annotations and then aggregated to one median value per recording and lead. The amplitude analyses used targeted samples, because retrieving voltages requires loading the corresponding raw signal files: 200 healthy recordings for the all-lead comparison and 100 recordings per class for the Figure 6 contrasts. Amplitudes were read as the raw signal voltage at each detected peak relative to a per-beat isoelectric baseline (median of the twelve samples ending at QRS onset). Each MedalCare-XL ECG is synthesised from common P-wave and QRST templates, while successive beats vary in RR timing and in the stretch or compression of the QRS-offset–T-offset segment; aggregating to one value per record matches the paper's per-ECG extraction. The computation was done in Python (pandas/NumPy); an equivalent MATLAB script, reproduce_paper_stats.m, is kept alongside the pipeline.

Timing. Five of the six healthy timing biomarkers reproduce the published simulated means to within single-digit-to-low-tens of milliseconds, QRS duration to 0.3 ms. The sixth, QT, reads +70.6 ms against the published Table 6 but is close to the descriptor's preprint (~385 ms; Section 4.6). This unresolved discrepancy should be interpreted as a difference between the references rather than proof that either value is correct. Under the per-record

aggregation, the standard deviations also match the paper reasonably closely: QRS duration is identical at 14.7 ms, while QT interval (23.8 ms), T-wave duration (27.7 ms), and RR interval (59.7 ms) remain close to the published values.

Feature	mean |Δμ| (ms)	mean Δμ (ms)	our σ (ms)	paper σ (ms)
P duration	6.1	−6.1	24.0	21.2
QRS duration	0.3	−0.3	14.7	14.7
T duration	1.8	−1.8	27.7	26.6
PQ interval	7.5	−7.5	34.3	30.1
QT interval	70.6	+70.6	23.8	24.5
RR interval	0.8	+0.8	59.7	54.4

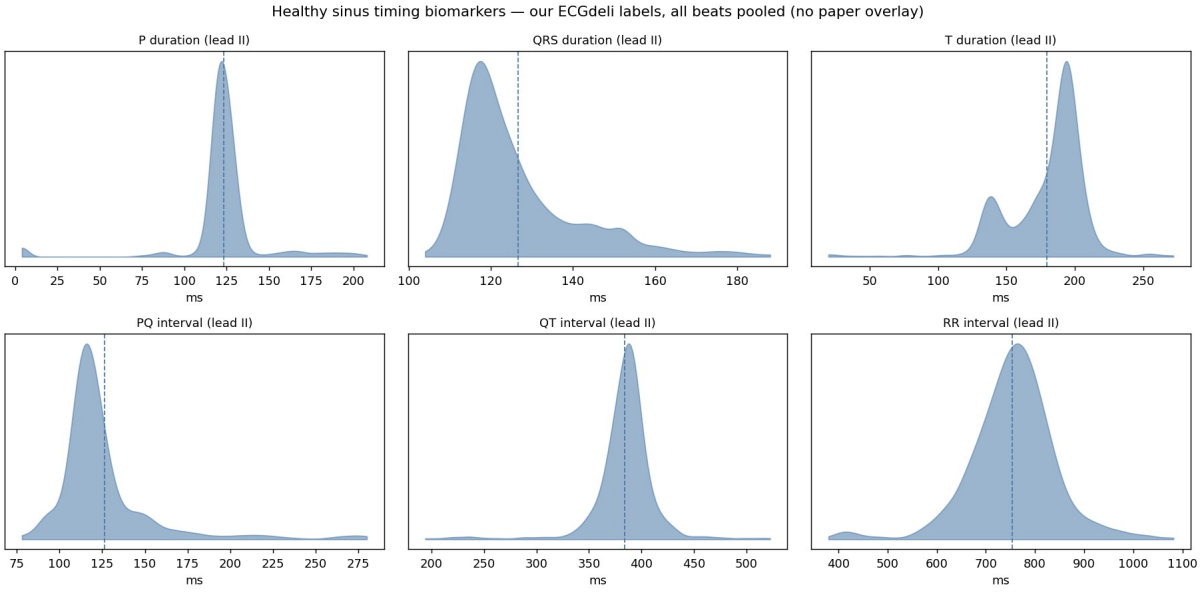
Amplitude. The five amplitude features reproduce the published values to within 0.19 mV (the maximum, on the R wave; all-lead mean absolute differences over a 200-record sample) and recover the expected lead-wise polarity (R positive in I/II/V5/V6, negative in aVR/V1–V3). Residual differences, largest for R, may reflect the baseline definition, lead-specific peak placement, or fiducial differences.

Feature	mean |Δ| (mV)	lead sign agreement
P amplitude	0.02	12/12
Q amplitude	0.05	11/12
R amplitude	0.19	11/12
S amplitude	0.03	12/12
T amplitude	0.06	12/12

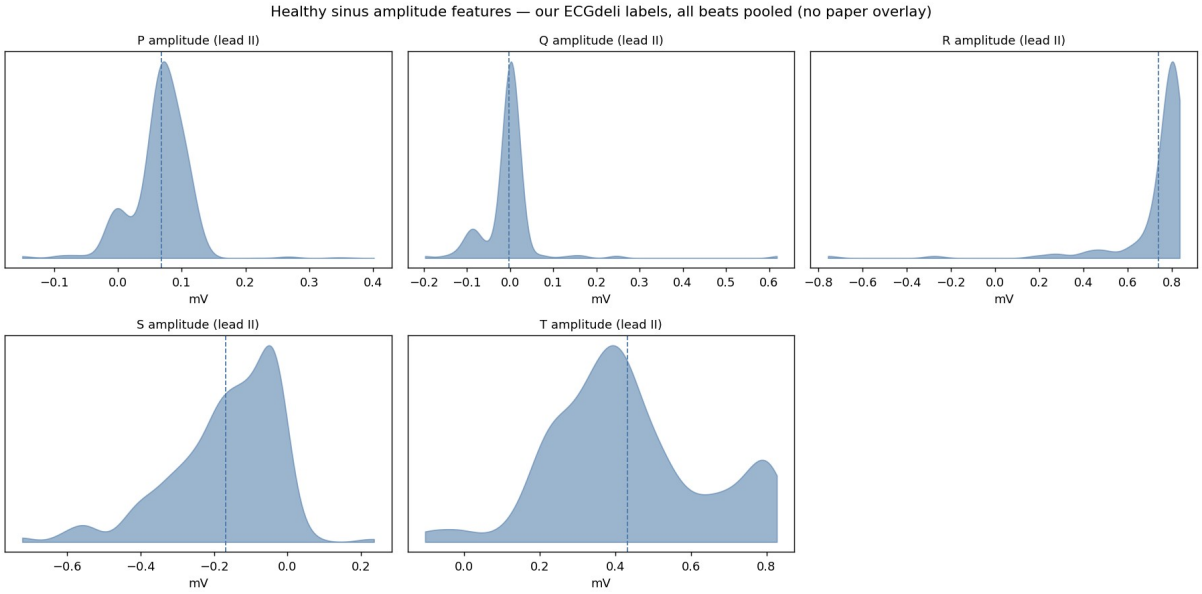
Per-class (Figure 6). Every disease class shifts in the physiologically expected direction: AV block prolongs PR by ~90 ms, LBBB widens QRS, LAE and IAB prolong the P wave, and MI lengthens QT. The labels therefore reproduce the reported disease-specific structure at the population level, but this does not establish beat-level label accuracy.

Class	ΔP dur	ΔQRS	ΔPQ	ΔQT	Expected pattern
AV block	+0.8	−2.7	+92.5	−1.7	PR prolongation
LBBB	−0.6	+10.5	+9.0	+4.0	QRS widening
RBBB	−6.2	+2.8	−8.2	+10.0	QRS widening
IAB	+6.9	+0.4	+15.0	+2.1	P / PQ prolongation
LAE	+16.6	−0.3	+24.0	+1.8	P-wave widening
FAM	+1.6	−0.6	+6.3	+0.2	mild P change
MI	+22.5	+2.4	+46.8	+19.2	QT / repolarisation change

Per-beat view (no comparison). Before the per-record aggregation, it is worth seeing the raw distributions of our labels with all beats pooled, exactly as first computed. The template-related beats produce narrow, sometimes multi-modal peaks whose shape does not match the paper's per-ECG statistics, so these are shown on their own, without the paper overlay. The per-record comparison follows.



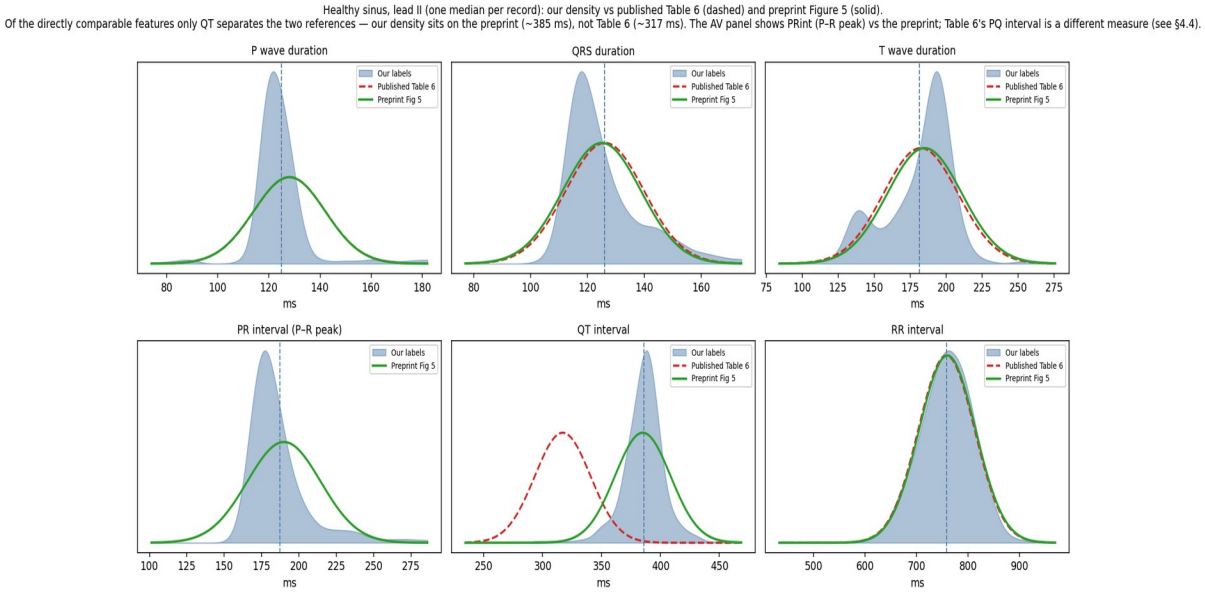
Healthy sinus timing biomarkers (lead II): our labels, all beats pooled, shown without a paper overlay, not directly comparable with the descriptor's per-ECG statistics.



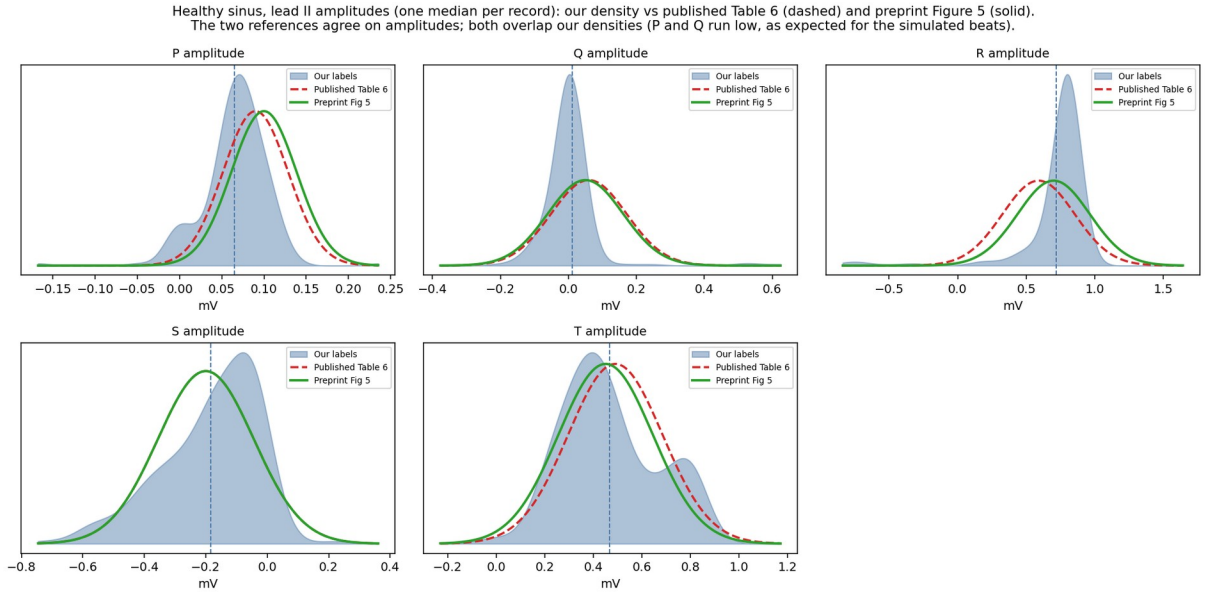
Healthy sinus amplitude features (lead II): our labels, all beats pooled, shown without a paper overlay.

Distributions. Overlaying the density of each healthy biomarker on the reference Gaussians shows broad agreement for several features, not exact equality of shape. Each panel carries the applicable references: the published Table 6 Gaussian (dashed) and the preprint Figure 5 (solid). Because the preprint publishes only probability-density plots (no numeric table), its curves are centred on the Figure 5 values with widths borrowed from the Table 6 standard deviation. Among the directly comparable timing features, P-wave duration, QRS duration, T-wave duration, and RR interval lie close to both reference curves, while QT separates the references. The atrioventricular timing panel instead

compares our PRint with the preprint only, because Table 6 reports the different PQint measure; the amplitudes broadly overlap both references.



Healthy timing biomarkers (lead II, one value per record): our label densities (shaded) vs the published Table 6 Gaussians (dashed red) and the preprint Figure 5 reference (solid green). Of the directly comparable features (P/QRS/T duration, RR) both references overlap our densities; only QT separates them, with our density on the preprint (~385 ms), not Table 6 (~317 ms). The fourth panel is the preprint's PR interval (P-onset to R-peak, ~190 ms); Table 6 instead reports the PQ interval (P-onset to QRS-onset, ~127 ms) — a different measure shown in the Section 4.4 table — so no Table 6 curve appears on that panel.



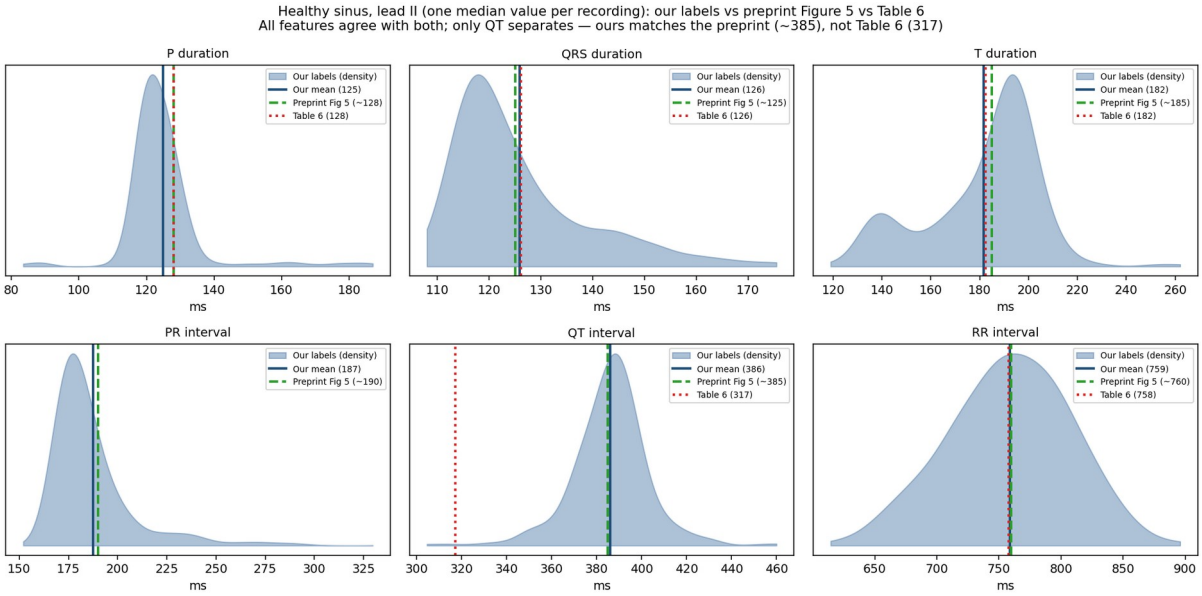
Healthy amplitude features (lead II, one value per record): our densities (shaded) vs the published Table 6 (dashed red) and preprint Figure 5 (solid green) references. The two references agree on the amplitudes, and both overlap our densities.

In short, the relabelling reproduces the MedalCare-XL published population statistics across timing means, spreads, amplitudes and disease-class structure, with QT closer to the descriptor's preprint (Section 4.6) than to the published Table 6. Because the paper's features were also produced by ECGdeli, this confirms reproducibility of the pipeline, not absolute accuracy, which is why the QT is compared against the preprint and cross-checked with NeuroKit2 below. Averaging ECGdeli with NeuroKit2 was also tested and moves every biomarker further from Table 6 (the paper's reference is itself ECGdeli-derived), so combining tools was not pursued.

4.6 Comparison against the preprint (per-signal aggregation)

A preprint of the descriptor (arXiv:2211.15997) reports the same feature distributions in its Figure 5. To compare at the population level, we summarise each recording to one value per lead, the median of each interval across its ~13 beats, and then take the cohort mean and spread per lead and class. The two levels are, per-signal median (Level 1), then per-lead/per-class summary (Level 2). Each MedalCare recording is a 10 s ECG whose beats share the P/QRS template and vary only in RR and a per-beat stretch of the QRSoff–Toff segment, so a per-recording median matches the paper's per-ECG comparison. The choice of the median specifically is ours (the paper does not state its beat aggregation strategy).

Four directly comparable non-QT timing features—P-wave duration, QRS duration, T-wave duration and RR interval—are close to both published references. The atrioventricular timing feature differs in definition between the two versions of the descriptor: the preprint reports PRint, measured from P-wave onset to the R peak, whereas the published Table 6 reports PQint, measured from P-wave onset to QRS onset. Our corresponding lead-II values are 187 ms for PRint, close to the preprint estimate of approximately 190 ms, and 124 ms for PQint, close to the published Table 6 value of 127 ms. QT remains the only directly comparable timing feature showing a large numerical discrepancy between the two references: our value of 386 ms is close to the preprint estimate of approximately 385 ms, but not to the published Table 6 value of 317 ms. Figure 6's per-class shifts also reproduce (wider QRS in RBBB/LBBB, prolonged PR in AV block, wider P in IAB/LAE, prolonged QT in MI), and the R/S/T amplitudes broadly match, while P and Q amplitudes are low in the relevant leads. An independent manual delineation of one ECG by the supervisor gave a lead-II QT of 364 ms, closer to our labels than to 317 ms, but this is one example and is not an independent ground truth.



Healthy sinus, lead II: per-recording median densities compared with the applicable reference values. P-wave duration, QRS duration, T-wave duration, and RR interval are directly comparable between the preprint and Table 6. The atrioventricular panel compares PRint with the preprint only, because Table 6 instead reports PQint. Among the directly comparable features, QT is the only substantial reference discrepancy, with our distribution closer to the preprint.

Per-class reproduction (Figure 6).

Beyond the healthy lead-II panels of Figure 5, we also reproduced the per-class contrasts of Figure 6 (disease vs healthy, in the disease-relevant lead). Every timing shift moves in the direction the figure shows: QRS duration widens in RBBB (+5 ms) and LBBB (+12 ms), the PR interval prolongs in AV block (+94 ms), P duration widens in IAB (+8 ms) and LAE (+11 ms), and QT prolongs in myocardial infarction at V4 (+12 ms). The amplitude panels agree too: MI changes the Q amplitude in lead II (-0.02 to +0.14 mV) and the R amplitude in V2 (-2.41 to -2.90 mV), while fibrotic atrial cardiomyopathy reduces the P-wave amplitude in aVL (0.043 to 0.008 mV) and V6 (0.076 to 0.060 mV); left atrial enlargement leaves the R amplitude unchanged, as expected for an atrial condition. So, the labels reproduce the reported population-level feature changes not only for the healthy population but also for the disease-specific contrasts that Figure 6 was designed to show. (Each Figure 6 amplitude contrast uses a 100-record sample per class; the healthy lead-II amplitude comparison in Section 4.5 uses 200 records, and its all-lead summary is in amplitudes_alllead_sinus.csv.)

4.7 Independent second-tool cross-check (NeuroKit2)

As a final, independent check, the ECGdeli labels were cross-checked against NeuroKit2, a separate open-source delineator that uses a discrete wavelet

method and shares no code with ECGdeli. The check was run over the entire dataset, all 16,848 records and all twelve leads. Each ECGdeli beat was matched one-to-one to its nearest NeuroKit2 beat by R-peak within tolerance (each NeuroKit2 beat is used at most once); because successive beats are separated by roughly an RR interval, far larger than the matching tolerance, re-running with strict one-to-one matching left every agreement figure unchanged. The number of matched beats varies by fiducial, from about 1.52 million (T-offset) to 2.00 million (R-peak). For each fiducial we recorded the fraction of matched beats on which the two tools agree within a per-fiducial tolerance, together with the median difference (NeuroKit2 minus ECGdeli).

Fiducial	Agree %	Median Δ (ms)
P onset	55.7 %	+26
P peak	64.9 %	+4
P offset	60.0 %	−20
QRS onset	21.1 %	−44
Q peak	30.6 %	−24
R peak	46.7 %	0
S peak	32.8 %	+2
QRS offset	44.0 %	−12
T onset	34.6 %	+56
T peak	**82.3 %**	**−2**
T offset	**81.5 %**	**−8**

Agreement is the share of matched beats within tolerance; a positive median means NeuroKit2 places the point later than ECGdeli. The T-peak and T-offset rows are highlighted. Because R-peaks were used for beat matching, the R-peak row is not an independent validation.

The T-end receives partial independent support. Across the whole dataset and all twelve leads, the two tools agree on the T-peak 82.3 % of the time and on the T-offset 81.5 % of the time within the stated tolerances, with median differences of −2 ms and −8 ms. This shows majority agreement around the T-peak and T-offset, but the medians do not describe the full spread and do not prove that each label is accurate. The result makes a gross systematic T-offset displacement of approximately 70 ms less likely, while the QT difference between Table 6 and the preprint remains unresolved.

Two fiducials show low agreement. NeuroKit2 places the QRS-onset ~44 ms earlier than ECGdeli, and the T-onset ~56 ms later. Because ECGdeli's QRS onset produces PQ, or clinical PR, intervals close to the published reference, it is retained as the project reference, but neither tool can be treated as ground truth. The T-onset is an ambiguous ST-to-T junction on which delineation conventions may differ. These offsets indicate systematic method differences and may include labelling error.

Both tools may differ when placing P and T boundaries or peaks where waves are small, inverted or biphasic, contributing to the lower agreement in those settings.

Isolating large inter-tool disagreements. To focus review beyond the systematic boundary offsets, we kept only beats on which an actual peak (P, R or T) or the T-offset differs by more than 60 ms. This leaves 608,176 beats — about 30 % of matched beats and they cluster where morphology is hardest

Disease class	Large-disagreement beats	Share
mi	392,650	64.6 %
rbbb	41,274	6.8 %
lae	31,725	5.2 %
iab	30,519	5.0 %
fam	28,774	4.7 %
lbbb	28,054	4.6 %
avblock	27,618	4.5 %
sinus	27,562	4.5 %

Nearly two-thirds of these large inter-tool disagreements are myocardial-infarction beats, consistent with MI's altered repolarisation and its biphasic and inverted T waves. By lead they concentrate in aVR (105,940), aVL (81,096), I (68,402) and III (56,641), the low-amplitude and inverted-wave leads, while lead II has the fewest (29,632). The split across P-peak (≈394k), T-offset (≈261k) and T-peak (≈239k) is consistent with small or atypical P and T waves in particular leads and in infarction. These beats will be prioritized for manual review or to down-weight in strict per-lead T-timing evaluation.

5.1 Quality-control status carried with each signal

So that signals needing manual correction are trackable directly from the label table, each row carries a qc_status derived from the per-record-lead analysis of Section 4.3, together with a persistent flag, the list of problem leads, and a review priority

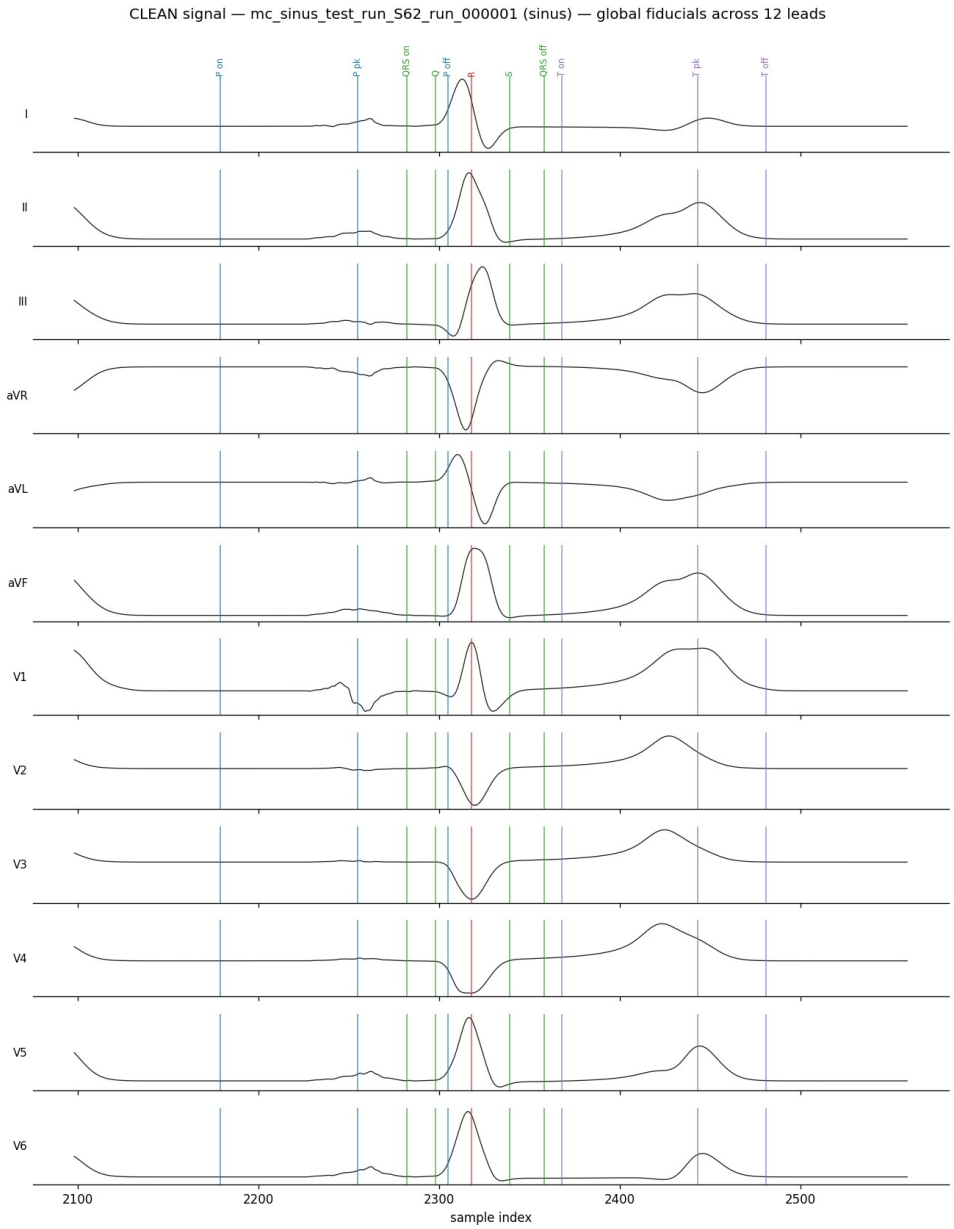
qc_status	Signals	Meaning / action
critical	2,300 (13.7%)	At least one lead is persistently flagged (>=50% of its beats flagged). These are prioritized for manual correction.
minor	5,512 (32.7%)	Some critical beats but no persistently flagged lead - lower-priority cases that an aligned per-record consensus may absorb.
clean	9,036 (53.6%)	No critical beats detected by the automated screen; retained subject to later checks.

The 2,300 critical signals correspond to the 4,251 persistently flagged (record, lead) units (a signal can have several problem leads) and are dominated by myocardial infarction (1,720), then LBBB (131) and RBBB (102). The higher-priority combined review set contains 13,792 record–lead units across 5,601 recordings: it comprises the 4,251 units that meet the persistent rule-based QC threshold and 9,541 additional units that fall below that threshold but contain both rule-based flags and persistent cross-method disagreement, captured in require_manual_label_priority.csv.

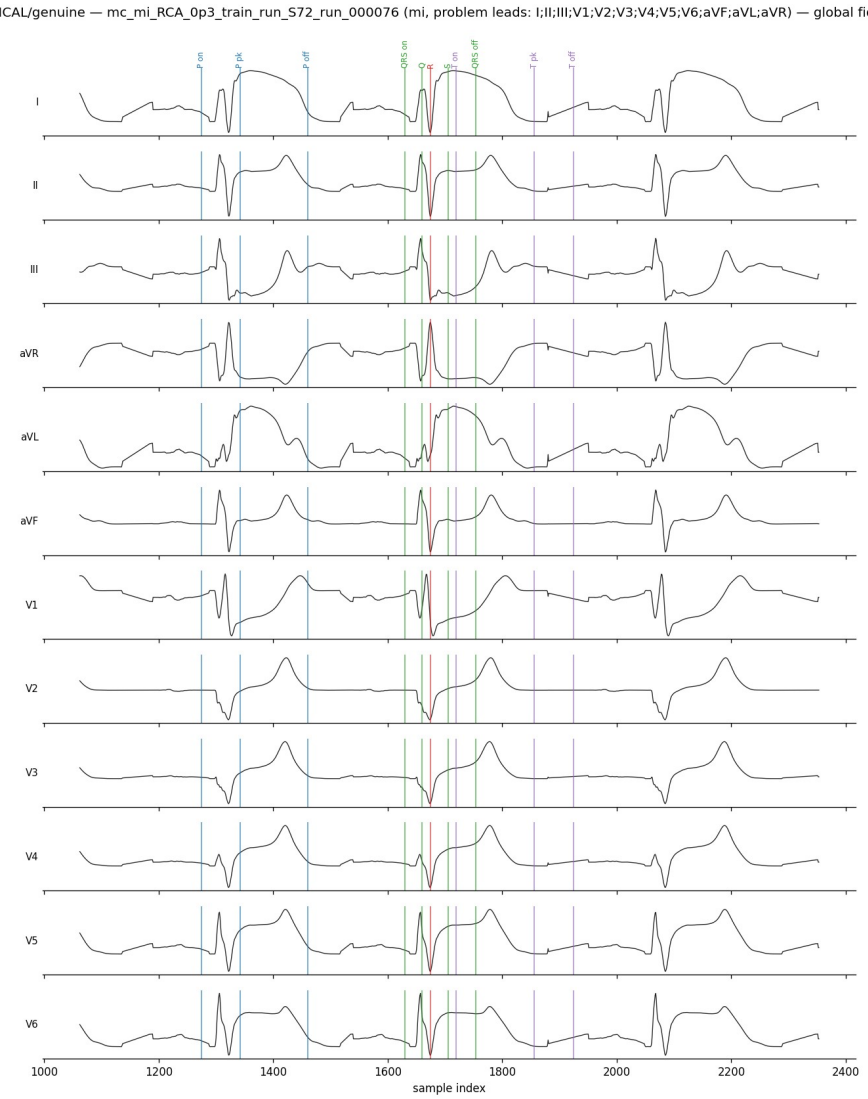
5.2 Worked examples - clean versus critical labels

The two figures below show the derived global fiducials drawn across the twelve leads for a clean signal and a critical one, which is also the exact view the manual-correction tool presents. In the clean healthy signal, the global fiducials provide a shared multilead timing estimate that is visually consistent with the dominant waveform boundaries across the leads; a single global line is not expected to coincide with every lead-specific boundary. This example shows that the clean and minor labels pass the automated structural and interval-based QC screen, not that every boundary is correct. The visible merge seam — the global QRS-onset sits marginally before the global P-offset because the two are drawn from different leads — is a consequence of the fusion convention and must be resolved when constructing non-overlapping dense targets.

In the critical myocardial infarction signal the labels visibly fail to match the abnormal morphology: an over-wide P wave is placed on an early deflection while the true QRS lies far to its right. Manual Correction will target such cases.



Clean healthy signal: global fiducials provide a shared multilead timing estimate that is visually consistent with the dominant waveform boundaries; the QRS-onset / P-offset merge seam remains visible.



Critical (persistently flagged) MI signal: the labels do not match the abnormal morphology (an over-wide P wave placed on an early deflection), the kind of case manual correction targets.

Appendix A — Part 1 quality-control code (qc_review_list.py)

The exact script used for Part 1. Thresholds (TOL, GROSS, QRS/QT/PR ranges) are configurable at the top.

```
#!/usr/bin/env python3

"""

qc_review_list.py  -  Flag low-quality ECGdeli beats for manual review.



Streams the ECGdeli fiducials CSV (works on the full 2.6M-row file) and writes:

  * medalcare_qc_review_list.csv      one row per CRITICAL (record, lead, beat) + reasons
```

```
  * medalcare_qc_record_summary.csv  one row per record (worst first)


Severity tiers (a small tolerance ignores trivial boundary wobble):
  CRITICAL  -> review/fix or exclude. Any of:
      - structural: QRS-internal order wrong  (QRSon<Q<R<S<QRSoff violated > TOL)
      - QRSdur / QT / PR outside physiological range
      - gross boundary inversion > GROSS samples
      - R peak missing
  MINOR     -> moderate P/T boundary inversion (TOL..GROSS samples); usually fine
  CLEAN     -> nothing, or inversion <= TOL samples (negligible)

Only CRITICAL beats go in the review list, so manual effort is targeted.

Usage:  python3 qc_review_list.py [fiducials.csv]
"""
import csv, sys, os
from collections import Counter, defaultdict

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = HERE
while ROOT != os.path.dirname(ROOT) and not os.path.isfile(os.path.join(ROOT,"config","paths.yaml")):
    ROOT = os.path.dirname(ROOT)
SRC = sys.argv[1] if len(sys.argv) > 1 else
os.path.join(ROOT,"ecgdeli_labelling","data","primary","medalcare_fiducials_ecgdeli.csv")
OUTDIR = os.path.join(ROOT,"ecgdeli_labelling","data","qc")
REVIEW = os.path.join(OUTDIR, "medalcare_qc_review_list.csv")
SUMMARY = os.path.join(OUTDIR, "medalcare_qc_record_summary.csv")

# ---- thresholds ----
TOL   = 3      # samples: ignore boundary inversions this small (<=6 ms @500Hz)
GROSS = 20     # samples: boundary inversion above this (>40 ms) is critical
QRS_MIN, QRS_MAX = 40, 200      # ms
QT_MIN,  QT_MAX  = 250, 700     # ms
```

```
PR_MIN,  PR_MAX  = 80, 400      # ms


ORDER = ["p_onset_sample","p_peak_sample","p_offset_sample","qrs_onset_sample","q_peak_sample",
         "r_peak_sample","s_peak_sample","qrs_offset_sample","t_onset_sample","t_peak_sample","t_offset_sample"]
LBL   = ["P_on","P_pk","P_off","QRSon","Q","R","S","QRSoff","T_on","T_pk","T_off"]
QRSset = {"QRSon","Q","R","S","QRSoff"}


def gi(r, k):
    v = r.get(k, ""); return int(v) if v not in ("", "None") else None


review = []
per_record = defaultdict(lambda: {"total": 0, "critical": 0, "minor": 0, "reasons": Counter(),
                                  "disease_class": "", "mi_subclass": "", "split": ""})
# per (record, lead): a MedalCare-XL ECG repeats a common P/QRST template with beat-level RR
# and repolarisation-timing variation, so the beats of a (record,lead) share one source morphology.
# A lone flagged beat among clean siblings is a transient delineation glitch; a unit is only a
# persistently flagged unit when a majority of its beats are flagged. Tracking this separates
# persistent issues from transient per-beat glitches.
per_rl = defaultdict(lambda: {"total": 0, "critical": 0, "disease_class": ""})
crit_tally = Counter()
total = nminor = 0


for r in csv.DictReader(open(SRC)):
    total += 1
    rec = r["record_id"]; pr_ = per_record[rec]
    rl_ = per_rl[(rec, r["lead"])]; rl_["total"] += 1; rl_["disease_class"] = r["disease_class"]
    pr_["total"] += 1
    pr_["disease_class"] = r["disease_class"]; pr_["mi_subclass"] = r.get("mi_subclass", ""); pr_["split"] = r["split"]
    fs = float(r["fs_hz"]); ms = 1000.0 / fs
    reasons = []


    seq = [(LBL[i], gi(r, k)) for i, k in enumerate(ORDER)]
```

```
seq = [(l, v) for l, v in seq if v is not None]
struct = False; gross = 0; mild = 0
for (la, va), (lb, vb) in zip(seq, seq[1:]):
    if va > vb:
        m = va - vb
        if la in QRSset and lb in QRSset and m > TOL:
            struct = True
        elif m > GROSS:
            gross = max(gross, m)
        elif m > TOL:
            mild = max(mild, m)
if struct: reasons.append("QRS_structural")
if gross:  reasons.append(f"gross_boundary:{gross}smp")


# biomarkers
qon, qoff = gi(r, "qrs_onset_sample"), gi(r, "qrs_offset_sample")
pon, toff = gi(r, "p_onset_sample"), gi(r, "t_offset_sample")
if qon is not None and qoff is not None:
    d = (qoff - qon) * ms
    if d < QRS_MIN or d > QRS_MAX: reasons.append(f"QRSdur={d:.0f}ms")
if qon is not None and toff is not None:
    d = (toff - qon) * ms
    if d < QT_MIN or d > QT_MAX: reasons.append(f"QT={d:.0f}ms")
if pon is not None and qon is not None:
    d = (qon - pon) * ms
    if d < PR_MIN or d > PR_MAX: reasons.append(f"PR={d:.0f}ms")
if r["qrs_present"] != "1" or gi(r, "r_peak_sample") is None:
    reasons.append("R_missing")


if reasons:                       # CRITICAL
    pr_["critical"] += 1; rl_["critical"] += 1
    for f in reasons:
        k = f.split(":")[0].split("=")[0]; pr_["reasons"][k] += 1; crit_tally[k] += 1
```

```
        review.append({"record_id": rec, "disease_class": r["disease_class"], "mi_subclass": r.get("mi_subclass", ""),
            "split": r["split"], "lead": r["lead"], "beat_id": r["beat_id"], "flags": ";".join(reasons),
            "p_onset_sample": r["p_onset_sample"], "p_offset_sample": r["p_offset_sample"],
            "qrs_onset_sample": r["qrs_onset_sample"], "r_peak_sample": r["r_peak_sample"],
            "qrs_offset_sample": r["qrs_offset_sample"], "t_peak_sample": r["t_peak_sample"], "t_offset_sample":
r["t_offset_sample"]})
    elif mild:                      # MINOR
        pr_["minor"] += 1; nminor += 1

# write review list
if review:
    with open(REVIEW, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=list(review[0].keys())); w.writeheader(); w.writerows(review)

# per-record summary, worst (by critical rate) first
summ = []
for rec, d in per_record.items():
    frac = d["critical"] / d["total"] if d["total"] else 0
    dom = d["reasons"].most_common(1)[0][0] if d["reasons"] else ""
    summ.append({"record_id": rec, "disease_class": d["disease_class"], "mi_subclass": d["mi_subclass"], "split": d["split"],
                 "beats_total": d["total"], "beats_critical": d["critical"], "beats_minor": d["minor"],
                 "frac_critical": round(frac, 4), "dominant_reason": dom})
summ.sort(key=lambda x: -x["frac_critical"])
with open(SUMMARY, "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=list(summ[0].keys())); w.writeheader(); w.writerows(summ)

ncrit = len(review)
print(f"beats total            : {total}")
print(f"CRITICAL (review these): {ncrit}  ({100*ncrit/total:.2f}%)  -> medalcare_qc_review_list.csv")
print(f"MINOR (small P/T wobble): {nminor}  ({100*nminor/total:.2f}%)  -> usually fine, left out of review list")
print(f"CLEAN                  : {total-ncrit-nminor}  ({100*(total-ncrit-nminor)/total:.2f}%)")
print(f"records                : {len(per_record)}    fully-clean records: {sum(1 for d in per_record.values() if d['critical']==0)}")
print("\ncritical-flag reasons:")
```

```
for k, v in crit_tally.most_common():
    print(f"  {k:16s}: {v}")
print("\ncritical rate by disease class:")
agg = defaultdict(lambda: [0, 0])
for d in per_record.values():
    agg[d["disease_class"]][0] += d["critical"]; agg[d["disease_class"]][1] += d["total"]
for c in sorted(agg, key=lambda k: -agg[k][0]/agg[k][1]):
    cr, to = agg[c]; print(f"  {c:10s}: {100*cr/to:5.1f}%  ({cr}/{to})")
print("\nworst 5 records to review first:")
for s in summ[:5]:
    print(f"  {s['record_id']:36s} {s['disease_class']:8s} {s['beats_critical']}/{s['beats_total']} ({100*s['frac_critical']:.0f}%) {s['dominant_reason']}")



# ---- per-(record,lead) view: separate persistently flagged units from transient per-beat glitches ----
# Because each ECG repeats one source template, the meaningful unit is (record, lead). A unit is a
# persistently flagged unit only if a majority (>=50%) of its beats are flagged; otherwise the flag is
# a transient glitch on an isolated beat of an otherwise-clean repeated morphology.
PERSIST_FRAC = 0.5
n_units = len(per_rl)
units_any  = sum(1 for d in per_rl.values() if d["critical"] > 0)
units_gen  = sum(1 for d in per_rl.values() if d["total"] and d["critical"]/d["total"] >= PERSIST_FRAC)
crit_in_gen = sum(d["critical"] for d in per_rl.values() if d["total"] and d["critical"]/d["total"] >= PERSIST_FRAC)
transient_share = 100*(1 - crit_in_gen/ncrit) if ncrit else 0
print("\nper-(record,lead) view  [each ECG repeats one source template, so this is the meaningful unit]:")
print(f"  (record,lead) units              : {n_units}")
print(f"  units with >=1 critical beat     : {units_any}  ({100*units_any/n_units:.2f}%)")
print(f"  persistently flagged units (>=50%): {units_gen}  ({100*units_gen/n_units:.2f}%)")
print(f"  transient share of critical beats: {transient_share:.0f}%  (isolated glitches a per-record consensus label would absorb)")
print("\npersistent-flag rate by disease class (per record-lead):")
gagg = defaultdict(lambda: [0, 0])
for d in per_rl.values():
    gen = 1 if (d["total"] and d["critical"]/d["total"] >= PERSIST_FRAC) else 0
    gagg[d["disease_class"]][0] += gen; gagg[d["disease_class"]][1] += 1
```

```
for c in sorted(gagg, key=lambda k: -gagg[k][0]/gagg[k][1]):

    g, to = gagg[c]; print(f"  {c:10s}: {100*g/to:5.2f}%  ({g}/{to})")
```

Appendix B — Preprocessing-sweep code

The two scripts used in Section 4.6. ecgdeli_config_sweep.m (MATLAB) relabels a small healthy sample under the four configurations; score_vs_table6.py scores any fiducials file against Table 6.

ecgdeli_config_sweep.m

```
%% ecgdeli_config_sweep.m
% Find the ECGdeli preprocessing config that best reproduces the paper's Table 6.
%
% Runs a small HEALTHY (sinus) sample under four preprocessing configs and writes
% one minimal fiducials CSV per config. Then score each in Python:
%     python3 score_vs_table6.py sweep_raw_iso.csv
%     python3 score_vs_table6.py sweep_filtered_iso.csv   ... etc.
% Keep the config with the smallest QT / total distance-to-paper, then set that
% SIGNAL_VER + APPLY_ISOLINE in run_ecgdeli_medalcare.m and relabel the full set.
%
% Requires ECGdeli on the path (same as the main driver).


clear; clc;
SCRIPT_DIR   = fileparts(mfilename('fullpath'));            % ecgdeli_labelling/scripts
REPO_ROOT    = fileparts(fileparts(SCRIPT_DIR));            % repo root (holds the signal files)
DATASET_ROOT = REPO_ROOT;
ECGDELI_PATH = fullfile(REPO_ROOT,'ECG_TOOL','ECGdeli');
MANIFEST     = fullfile(REPO_ROOT,'ecgdeli_labelling','data','input','medalcare_manifest.csv');
OUTDIR       = fullfile(REPO_ROOT,'ecgdeli_labelling','logs');   % diagnostic sweep_*.csv scratch
NREC = 30;          % sinus records per config (enough for stable per-lead means)
FS   = 500;

addpath(genpath(ECGDELI_PATH));
assert(exist('Annotate_ECG_Multi','file')==2,'ECGdeli not on path.');

T = readtable(MANIFEST,'Delimiter',',','TextType','string');
for vn = string(T.Properties.VariableNames)
    if isstring(T.(vn)), T.(vn) = fillmissing(T.(vn),'constant",""); end
end
sin = T(T.disease_class=="sinus",:);
sin = sin(1:min(NREC,height(sin)),:);
LEADS = ["I","II","III","aVR","aVL","aVF","V1","V2","V3","V4","V5","V6"];

% name , signal-version column , apply isoline?
configs = { 'raw_iso',       'path_raw',      true ; ...
            'raw_noiso',     'path_raw',      false; ...
            'filtered_iso',  'path_filtered', true ; ...
            'filtered_noiso','path_filtered', false };

for ci = 1:size(configs,1)
    name = configs{ci,1}; vercol = configs{ci,2}; useiso = configs{ci,3};
    outcsv = fullfile(OUTDIR, "sweep_"+name+".csv");
    fid = fopen(outcsv,'w');
    fprintf(fid,['record_id,disease_class,lead,beat_id,p_onset_sample,p_offset_sample,' ...
                 'qrs_onset_sample,qrs_offset_sample,t_onset_sample,t_offset_sample,r_peak_sample\n']);
    for k = 1:height(sin)
        rid = sin.record_id(k);
        fp  = fullfile(DATASET_ROOT, sin.(char(vercol))(k));
        try
            M = readmatrix(fp); if size(M,1)==12, sig = M.'; else, sig = M; end
```

```
            sig = double(sig);
            if useiso, try sig = Isoline_Correction(sig); catch; end; end
            [~, FPT_Cell] = Annotate_ECG_Multi(sig, FS, 'PQRST');
            if isempty(FPT_Cell), continue; end
            for L = 1:numel(LEADS)
                f = FPT_Cell{L}; if isempty(f), continue; end
                for b = 1:size(f,1)
                    g = @(c) col0(f,b,c);
                    fprintf(fid,'%s,sinus,%s,%d,%s,%s,%s,%s,%s,%s,%s\n', rid, LEADS(L), b, ...
                        g(1),g(3),g(4),g(8),g(10),g(12),g(6));   % Pon,Poff,QRSon,QRSoff,Ton,Toff,R
                end
            end
        catch ME
            fprintf('  %s (%s): %s\n', rid, name, ME.message);
        end
    end
    fclose(fid);
    fprintf('wrote %s\n', outcsv);
end
fprintf(['\nScore each config:\n' ...
         '  python3 score_vs_table6.py sweep_raw_iso.csv\n' ...
         '  python3 score_vs_table6.py sweep_raw_noiso.csv\n' ...
         '  python3 score_vs_table6.py sweep_filtered_iso.csv\n' ...
         '  python3 score_vs_table6.py sweep_filtered_noiso.csv\n' ...
         'Pick the smallest QT / total distance, then set SIGNAL_VER + APPLY_ISOLINE\n' ...
         'in run_ecgdeli_medalcare.m to match and relabel the full dataset.\n']);


function s = col0(f,b,c)   % FPT value -> 0-based string, "" if missing
    if c > size(f,2), s = ""; return; end
    x = f(b,c);
    if isnan(x) || x <= 0, s = ""; else, s = string(round(x-1)); end
end
```

```
#!/usr/bin/env python3
"""
score_vs_table6.py  -  Score any ECGdeli fiducials file against the paper's Table 6.

Extracts the six healthy-cohort timing biomarkers (and, if signals are given, the
five amplitudes) per lead from a fiducials CSV, compares the per-lead means to the
Table 6 'sim' column, and prints a single distance-to-paper score. Run it on the
output of each ECGdeli preprocessing config (see ecgdeli_config_sweep.m) and keep
the config with the smallest score.

Usage:
    python3 score_vs_table6.py FIDUCIALS.csv [--class sinus] [--signals /path/to/MedalCareXL Data]
"""
import csv, sys, argparse, os
import numpy as np
from collections import defaultdict

# Table 6 'sim' (healthy) means per lead
T6_TIMING = {  # [Pdur,QRSdur,Tdur,PQint,QTint,RRint]
 "I":[124.06,131.31,178.12,128.07,310.54,758.15],"II":[128.09,126.10,182.33,127.18,317.08,758.02],
 "III":[164.52,126.80,183.16,171.88,306.94,757.99],"aVR":[127.42,128.81,179.38,126.19,318.73,757.97],
 "aVL":[154.50,128.62,182.76,169.37,299.19,758.08],"aVF":[141.00,125.02,184.05,142.60,310.43,758.06],
 "V1":[140.78,129.05,180.72,160.57,303.89,758.06],"V2":[155.93,136.12,176.15,181.65,287.71,757.90],
 "V3":[154.23,132.80,179.01,174.20,285.50,758.01],"V4":[140.60,127.32,180.05,148.55,290.09,758.03],
 "V5":[128.69,123.89,177.39,126.73,310.74,758.12],"V6":[123.44,126.55,174.63,118.63,320.51,758.06]}
TFEAT=["Pdur","QRSdur","Tdur","PQint","QTint","RRint"]
LEADS=list(T6_TIMING)

ap=argparse.ArgumentParser()
```

```
ap.add_argument("fiducials"); ap.add_argument("--klass",default="sinus")
ap.add_argument("--signals",default=None,help="dataset root (enables amplitude scoring)")
ap.add_argument("--manifest",default="ecgdeli_labelling/data/input/medalcare_manifest.csv")
a=ap.parse_args()

def gi(v): return int(v) if v not in ("","None") else None
data=defaultdict(list)
hdr=None
for i,r in enumerate(csv.DictReader(open(a.fiducials))):
    if r.get("disease_class")!=a.klass: continue
    g=lambda k:gi(r.get(k+"_sample",""))
    data[(r["record_id"],r["lead"])].append((int(r["beat_id"]),g("p_onset"),g("p_offset"),
        g("qrs_onset"),g("qrs_offset"),g("t_onset"),g("t_offset"),g("r_peak")))
ms=2.0
ours={ld:{f:[] for f in TFEAT} for ld in LEADS}
for (rid,ld),bl in data.items():
    if ld not in LEADS: continue
    bl.sort(); Rs=[b[7] for b in bl if b[7] is not None]
    for i in range(1,len(Rs)): ours[ld]["RRint"].append((Rs[i]-Rs[i-1])*ms)
    for (_,pon,poff,qon,qoff,ton,toff,rp) in bl:
        if pon and poff: ours[ld]["Pdur"].append((poff-pon)*ms)
        if qon and qoff: ours[ld]["QRSdur"].append((qoff-qon)*ms)
        if ton and toff: ours[ld]["Tdur"].append((toff-ton)*ms)
        if pon and qon: ours[ld]["PQint"].append((qon-pon)*ms)
        if qon and toff: ours[ld]["QTint"].append((toff-qon)*ms)

print(f"scoring {a.fiducials}  (class={a.klass})\n")
print(f"{'feature':8s} {'mean|Δ| (ms)':>14s}")
total=0; nfeat=0
for j,f in enumerate(TFEAT):
    ds=[]
    for ld in LEADS:
        if ours[ld][f]: ds.append(abs(np.mean(ours[ld][f])-T6_TIMING[ld][j]))
    if ds:
        md=np.mean(ds); print(f"{f:8s} {md:12.1f}"); total+=md; nfeat+=1
print(f"\nTIMING distance-to-paper (mean of the six mean|Δ|): {total/nfeat:.1f} ms")
print("(lower is closer; QT dominates when T-end is misaligned)")
```

Appendix C — Second-tool cross-check code (crosscheck_neurokit.py)

The script used for Section 4.7. Configured for the full run (all records, all 12 leads).

```
#!/usr/bin/env python3
"""
crosscheck_neurokit.py  -  Independent second-tool verification of the ECGdeli labels.

Runs NeuroKit2 (an independent DWT delineator) and compares its P/QRS/T fiducials
against the ECGdeli labels, beat by beat, matched on the R-peak. Agreement between the
two tools provides convergent supporting evidence, while disagreement beyond tolerance
identifies cases requiring further inspection; neither tool is treated as ground truth.
Disagreeing beats are written to a disagreement list for manual review.

FULL-RUN configuration:
    N_PER_CLASS = None   -> every record (all 16,848), not a sample
    ALL_LEADS   = True   -> all 12 leads (else lead II only)
Expect a long run (all records x 12 leads is ~200k delineations). Progress prints
every 200 records so you can see it is alive. It is safe to reduce ALL_LEADS to
False (lead II only) for a much faster but still complete-record verification.

Run:
    pip install neurokit2 numpy
    python3 crosscheck_neurokit.py

Outputs (this folder):
    consensus_agreement_summary.txt   per-fiducial agreement + median/mean offset
    consensus_disagreements.csv       every beat where the tools differ > tolerance (with lead)
"""
import os, csv, sys, time
import numpy as np
try:
    import neurokit2 as nk
except ImportError:
    sys.exit("Install NeuroKit2 first:  pip install neurokit2")

# ---------------- CONFIG ----------------
HERE      = os.path.dirname(os.path.abspath(__file__))
ROOT = HERE
while ROOT != os.path.dirname(ROOT) and not os.path.isfile(os.path.join(ROOT,"config","paths.yaml")):
    ROOT = os.path.dirname(ROOT)
DATA_ROOT = ROOT                                    # repo root (for signal path_raw)
MANIFEST  = os.path.join(ROOT,"ecgdeli_labelling","data","input","medalcare_manifest.csv")
FIDUCIALS = os.path.join(ROOT,"ecgdeli_labelling","data","primary","medalcare_fiducials_ecgdeli.csv")
XSUM      = os.path.join(ROOT,"ecgdeli_labelling","data","neurokit_crosscheck","summaries")
XINT      = os.path.join(ROOT,"ecgdeli_labelling","data","neurokit_crosscheck","intermediates")
N_PER_CLASS = None            # None = ALL records; or an int for a sample per class
ALL_LEADS   = True            # True = all 12 leads; False = lead II only
SIGNAL_VER  = "raw"
METHOD      = "dwt"           # wavelet (DWT) delineator. The non-wavelet check lives in crosscheck_neurokit_peak.py
FS          = 500
LEADS = ["I","II","III","aVR","aVL","aVF","V1","V2","V3","V4","V5","V6"]
LEAD_ROW = {l:i for i,l in enumerate(LEADS)}
TOL_MS = {"p_onset":40,"p_peak":25,"p_offset":40,"qrs_onset":25,"q_peak":20,"r_peak":15,
          "s_peak":20,"qrs_offset":25,"t_onset":50,"t_peak":30,"t_offset":50}
leads_run = LEADS if ALL_LEADS else ["II"]
leads_set = set(leads_run)

NK_KEY = {"p_onset":"ECG_P_Onsets","p_peak":"ECG_P_Peaks","p_offset":"ECG_P_Offsets",
          "qrs_onset":"ECG_R_Onsets","q_peak":"ECG_Q_Peaks","r_peak":"ECG_R_Peaks","s_peak":"ECG_S_Peaks",
          "qrs_offset":"ECG_R_Offsets","t_onset":"ECG_T_Onsets","t_peak":"ECG_T_Peaks","t_offset":"ECG_T_Offsets"}
```

```
FIDS = list(TOL_MS.keys())

def gi(v): return int(v) if v not in ("","None",None) else None

# --- 1. records to process ---
from collections import defaultdict
byc = defaultdict(list)
for r in csv.DictReader(open(MANIFEST)):
    byc[r["disease_class"]].append(r)
sample = {}
for c, lst in byc.items():
    for r in (lst if N_PER_CLASS is None else lst[:N_PER_CLASS]):
        sample[r["record_id"]] = (os.path.join(DATA_ROOT, r["path_"+SIGNAL_VER]), c)
ids = set(sample)
print(f"records to process: {len(ids)}  |  leads: {len(leads_run)}  |  {'FULL' if N_PER_CLASS is None else N_PER_CLASS} per
class")

# --- 2. ECGdeli fiducials for those records/leads ---
edeli = defaultdict(list)   # (rid,lead) -> list of beat dicts
for r in csv.DictReader(open(FIDUCIALS)):
    if r["lead"] in leads_set and r["record_id"] in ids:
        edeli[(r["record_id"], r["lead"])].append({k: gi(r.get(k+"_sample","")) for k in FIDS})
print(f"loaded ECGdeli fiducials for {len(edeli)} (record,lead) pairs")

# --- 3+4. run NeuroKit2, match beats by R-peak, compare ---
deltas = {k: [] for k in FIDS}
agree  = {k: [0, 0] for k in FIDS}
disagreements = []
done = 0; t0 = time.time()
for rid, (path, cls) in sample.items():
    try:
        M = np.loadtxt(path, delimiter=",")
    except Exception:
        continue
    for ld in leads_run:
        if (rid, ld) not in edeli:
            continue
        try:
            ecg = nk.ecg_clean(M[LEAD_ROW[ld], :], sampling_rate=FS)
            _, rp = nk.ecg_peaks(ecg, sampling_rate=FS)
            rpk = rp["ECG_R_Peaks"]
            _, waves = nk.ecg_delineate(ecg, rpeaks=rpk, sampling_rate=FS, method=METHOD)
        except Exception:
            continue
        nkR = list(rpk)
        nk_beats = []
        for i in range(len(nkR)):
            beat = {"r_peak": int(nkR[i]) if nkR[i] == nkR[i] else None}
            for k in FIDS:
                if k == "r_peak": continue
                arr = waves.get(NK_KEY[k], [])
                v = arr[i] if i < len(arr) else np.nan
                beat[k] = int(v) if (v == v and v is not None) else None
            nk_beats.append(beat)
        used = set()                        # enforce one-to-one: each NeuroKit beat matches once
        for eb in edeli[(rid, ld)]:
            er = eb["r_peak"]
            if er is None or not nk_beats: continue
            cand = [(abs(nk_beats[i]["r_peak"] - er), i) for i in range(len(nk_beats))
                    if i not in used and nk_beats[i]["r_peak"] is not None]
            if not cand: continue
            dist, bi = min(cand)
            if dist > 30:  # 60 ms
                continue
```

```
            used.add(bi); nb = nk_beats[bi]
            rowdis = {"record_id": rid, "disease_class": cls, "lead": ld, "r_peak": er}
            flagged = False
            for k in FIDS:
                if eb[k] is None or nb[k] is None: continue
                d = nb[k] - eb[k]; deltas[k].append(d); agree[k][1] += 1
                if abs(d) * (1000/FS) <= TOL_MS[k]:
                    agree[k][0] += 1
                else:
                    rowdis[k+"_ecgdeli"] = eb[k]; rowdis[k+"_neurokit"] = nb[k]; rowdis[k+"_delta_ms"] = round(d*1000/FS,1)
                    flagged = True
            if flagged:
                disagreements.append(rowdis)
    done += 1
    if done % 200 == 0:
        el = time.time()-t0
        print(f"  {done}/{len(ids)} records  ({el:.0f}s, ~{el/done*len(ids)/60:.0f} min total)")

# --- 5. write outputs ---
with open(os.path.join(XSUM, "consensus_agreement_summary.txt"), "w") as f:
    def line(s=""): print(s); f.write(s+"\n")
    scope = f"{len(leads_run)} lead(s), {done} records"
    line(f"NeuroKit2 ({METHOD}) vs ECGdeli  -  {scope}\n")
    line(f"{'fiducial':11s} {'agree%':>7s} {'n':>8s} {'medianΔ':>8s} {'meanΔ':>7s}  (ms)")
    for k in FIDS:
        w, n = agree[k]
        if n == 0: line(f"{k:11s} {'--':>7s} {0:>8d}"); continue
        d = np.array(deltas[k]) * (1000/FS)
        line(f"{k:11s} {100*w/n:6.1f}% {n:>8d} {np.median(d):7.1f} {d.mean():6.1f}")
    line(f"\nbeats with >=1 disagreement: {len(disagreements)}  -> consensus_disagreements.csv")

if disagreements:
    keys = sorted({k for d in disagreements for k in d})
    with open(os.path.join(XINT, "consensus_disagreements.csv"), "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=keys); w.writeheader(); w.writerows(disagreements)
print("done. See consensus_agreement_summary.txt")
```

Appendix D — Non-wavelet cross-check code (crosscheck_neurokit_peak.py)

The script used for Section 4.8. Non-wavelet (derivative) delineator; configured for lead II across every record.

```
#!/usr/bin/env python3

"""

crosscheck_neurokit_peak.py  -  NON-WAVELET third-opinion verification of the ECGdeli labels.



This is the sibling of crosscheck_neurokit.py. That script uses NeuroKit2's DWT

(wavelet) delineator; ECGdeli is also wavelet-based, so the two agreeing cannot

rule out a bias the wavelet family shares. This script instead runs NeuroKit2's

"peak" delineator - a DERIVATIVE / local-extrema method from a different algorithm

family (no wavelets) - and compares it to the ECGdeli labels the same way. Three
```

```
independent method families (ECGdeli-wavelet, NeuroKit-wavelet, NeuroKit-derivative)
agreeing on a fiducial is far stronger evidence than two wavelet tools agreeing.

The peak method returns fewer fiducials than DWT: the P/Q/S/T PEAKS, the P-onset
and the T-offset. It does NOT return the QRS onset/offset or the T-onset (those
boundaries are wavelet-only in NeuroKit2). That is fine here - the T-offset (the
one biomarker where we differ from the paper) IS returned, so this check
triangulates exactly the fiducial that matters most.

Configuration (kept light on purpose - this is a method-family check, not a full
per-beat consensus, so lead II across every record is sufficient and fast):
    N_PER_CLASS = None   -> every record (all 16,848)
    ALL_LEADS   = False  -> lead II only (set True for all 12 leads, much slower)

Run:
    pip install neurokit2 numpy
    python3 crosscheck_neurokit_peak.py

Outputs (this folder, named distinctly so they never collide with the DWT run):
    consensus_peak_agreement_summary.txt    per-fiducial agreement + median/mean offset
    consensus_peak_disagreements.csv        every beat where the tools differ > tolerance (with lead)
"""
import os, csv, sys, time
import numpy as np
try:
    import neurokit2 as nk
except ImportError:
    sys.exit("Install NeuroKit2 first:  pip install neurokit2")

# ---------- CONFIG ----------
HERE      = os.path.dirname(os.path.abspath(__file__))
ROOT = HERE
```

```
while ROOT != os.path.dirname(ROOT) and not os.path.isfile(os.path.join(ROOT,"config","paths.yaml")):
    ROOT = os.path.dirname(ROOT)
DATA_ROOT = ROOT                                        # repo root (for signal path_raw)
MANIFEST  = os.path.join(ROOT,"ecgdeli_labelling","data","input","medalcare_manifest.csv")
FIDUCIALS = os.path.join(ROOT,"ecgdeli_labelling","data","primary","medalcare_fiducials_ecgdeli.csv")
XSUM      = os.path.join(ROOT,"ecgdeli_labelling","data","neurokit_crosscheck","summaries")
XINT      = os.path.join(ROOT,"ecgdeli_labelling","data","neurokit_crosscheck","intermediates")
N_PER_CLASS = None            # None = ALL records; or an int for a sample per class
ALL_LEADS   = False           # False = lead II only (recommended); True = all 12 leads
SIGNAL_VER  = "raw"
METHOD      = "peak"          # NON-wavelet (derivative / local-extrema) delineator
FS          = 500
LEADS = ["I","II","III","aVR","aVL","aVF","V1","V2","V3","V4","V5","V6"]
LEAD_ROW = {l:i for i,l in enumerate(LEADS)}

# Only the fiducials the peak method actually returns (peaks + P-onset + T-offset)
TOL_MS = {"p_onset":40,"p_peak":25,"q_peak":20,"r_peak":15,"s_peak":20,"t_peak":30,"t_offset":50}
NK_KEY = {"p_onset":"ECG_P_Onsets","p_peak":"ECG_P_Peaks","q_peak":"ECG_Q_Peaks",
          "r_peak":"ECG_R_Peaks","s_peak":"ECG_S_Peaks","t_peak":"ECG_T_Peaks","t_offset":"ECG_T_Offsets"}
FIDS = list(TOL_MS.keys())

leads_run = LEADS if ALL_LEADS else ["II"]
leads_set = set(leads_run)

def gi(v): return int(v) if v not in ("","None",None) else None

# --- 1. records to process ---
from collections import defaultdict
byc = defaultdict(list)
for r in csv.DictReader(open(MANIFEST)):
    byc[r["disease_class"]].append(r)
sample = {}
```

```
for c, lst in byc.items():
    for r in (lst if N_PER_CLASS is None else lst[:N_PER_CLASS]):
        sample[r["record_id"]] = (os.path.join(DATA_ROOT, r["path_"+SIGNAL_VER]), c)
ids = set(sample)
print(f"[peak/non-wavelet] records: {len(ids)}  |  leads: {len(leads_run)}  |  {'FULL' if N_PER_CLASS is None else N_PER_CLASS} per class")


# --- 2. ECGdeli fiducials for those records/leads ---
edeli = defaultdict(list)   # (rid,lead) -> list of beat dicts
for r in csv.DictReader(open(FIDUCIALS)):
    if r["lead"] in leads_set and r["record_id"] in ids:
        edeli[(r["record_id"], r["lead"])].append({k: gi(r.get(k+"_sample","")) for k in (FIDS+["r_peak"])})
print(f"loaded ECGdeli fiducials for {len(edeli)} (record,lead) pairs")


# --- 3+4. run NeuroKit2 (peak method), match beats by R-peak, compare ---
deltas = {k: [] for k in FIDS}
agree  = {k: [0, 0] for k in FIDS}
disagreements = []
done = 0; t0 = time.time()
for rid, (path, cls) in sample.items():
    try:
        M = np.loadtxt(path, delimiter=",")
    except Exception:
        continue
    for ld in leads_run:
        if (rid, ld) not in edeli:
            continue
        try:
            ecg = nk.ecg_clean(M[LEAD_ROW[ld], :], sampling_rate=FS)
            _, rp = nk.ecg_peaks(ecg, sampling_rate=FS)
            rpk = rp["ECG_R_Peaks"]
            _, waves = nk.ecg_delineate(ecg, rpeaks=rpk, sampling_rate=FS, method=METHOD)
        except Exception:
```

```
    continue
nkR = list(rpk)
nk_beats = []
for i in range(len(nkR)):
    beat = {"r_peak": int(nkR[i]) if nkR[i] == nkR[i] else None}
    for k in FIDS:
        if k == "r_peak": continue
        arr = waves.get(NK_KEY[k], [])
        v = arr[i] if i < len(arr) else np.nan
        beat[k] = int(v) if (v == v and v is not None) else None
    nk_beats.append(beat)
used = set()                        # enforce one-to-one: each NeuroKit beat matches once
for eb in edeli[(rid, ld)]:
    er = eb["r_peak"]
    if er is None or not nk_beats: continue
    cand = [(abs(nk_beats[i]["r_peak"] - er), i) for i in range(len(nk_beats))
            if i not in used and nk_beats[i]["r_peak"] is not None]
    if not cand: continue
    dist, bi = min(cand)
    if dist > 30:  # 60 ms
        continue
    used.add(bi); nb = nk_beats[bi]
    rowdis = {"record_id": rid, "disease_class": cls, "lead": ld, "r_peak": er}
    flagged = False
    for k in FIDS:
        if k == "r_peak" or eb.get(k) is None or nb.get(k) is None: continue
        d = nb[k] - eb[k]; deltas[k].append(d); agree[k][1] += 1
        if abs(d) * (1000/FS) <= TOL_MS[k]:
            agree[k][0] += 1
        else:
            rowdis[k+"_ecgdeli"] = eb[k]; rowdis[k+"_neurokit"] = nb[k]; rowdis[k+"_delta_ms"] = round(d*1000/FS,1)
            flagged = True
    if flagged:
```

```
            disagreements.append(rowdis)
    done += 1
    if done % 200 == 0:
        el = time.time()-t0
        print(f"  {done}/{len(ids)} records  ({el:.0f}s, ~{el/done*len(ids)/60:.0f} min total)")


# --- 5. write outputs (distinct peak-tagged names) ---
with open(os.path.join(XSUM, "consensus_peak_agreement_summary.txt"), "w") as f:
    def line(s=""): print(s); f.write(s+"\n")
    scope = f"{len(leads_run)} lead(s), {done} records"
    line(f"NeuroKit2 (peak / NON-wavelet) vs ECGdeli  -  {scope}\n")
    line("peak method returns peaks + P-onset + T-offset only (no QRS on/off, no T-onset)\n")
    line(f"{'fiducial':11s} {'agree%':>7s} {'n':>8s} {'medianD':>8s} {'meanD':>7s}  (ms)")
    for k in FIDS:
        w, n = agree[k]
        if n == 0: line(f"{k:11s} {'--':>7s} {0:>8d}"); continue
        d = np.array(deltas[k]) * (1000/FS)
        line(f"{k:11s} {100*w/n:6.1f}% {n:>8d} {np.median(d):7.1f} {d.mean():6.1f}")
    line(f"\nbeats with >=1 disagreement: {len(disagreements)}  -> consensus_peak_disagreements.csv")


if disagreements:
    keys = sorted({k for d in disagreements for k in d})
    with open(os.path.join(XINT, "consensus_peak_disagreements.csv"), "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=keys); w.writeheader(); w.writerows(disagreements)
print("done. See consensus_peak_agreement_summary.txt")
```

Appendix E — Per-signal population-statistics code (build_per_signal_stats.py)

```
#!/usr/bin/env python3
"""
build_per_signal_stats.py  -  Two-level population statistics for MedalCare-XL, compared
to the paper (published Table 6, exact) and the PREPRINT Figure 5 (lead II, estimated).
```

```
Level 1 (per-signal): collapse each recording's ~13 beats to ONE median per (record, lead)
                      for each interval  -> per_signal_median.csv
Level 2 (per-lead/class): mean, SD, n across recordings, per lead, per class
                      -> per_lead_class_summary.csv
Comparison: sinus per-lead means vs Table 6 (exact) and, for lead II, vs preprint Fig 5
                      -> comparison_sinus_vs_paper.csv


Uses the ECGdeli interval columns already stored in the master CSV.
Run:  python3 build_per_signal_stats.py
"""
import pandas as pd, numpy as np, os
HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = HERE
while ROOT != os.path.dirname(ROOT) and not os.path.isfile(os.path.join(ROOT,"config","paths.yaml")):
    ROOT = os.path.dirname(ROOT)
DATA = os.path.join(ROOT,"statistics","data")
FID  = os.path.join(ROOT, "ecgdeli_labelling","data","primary","medalcare_fiducials_ecgdeli.csv")
LEADS = ["I","II","III","aVR","aVL","aVF","V1","V2","V3","V4","V5","V6"]

# master-CSV interval columns -> our feature names
COLMAP = {"Pdur":"ecgdeli_pdur_ms","QRSdur":"ecgdeli_qrsdur_ms","Tdur":"ecgdeli_tdur_ms",
          "PQint":"ecgdeli_pq_ms","PRint":"ecgdeli_pr_ms","QTint":"qt_interval_ms",
          "QTpeak":"qt_peak_ms","RRint":"ecgdeli_rr_ms"}
FEATS = list(COLMAP)

# Published Table 6 'sim' means (lead-wise): [Pdur,QRSdur,Tdur,PQint,QTint,RRint]
T6 = {"I":[124.06,131.31,178.12,128.07,310.54,758.15],"II":[128.09,126.10,182.33,127.18,317.08,758.02],
 "III":[164.52,126.80,183.16,171.88,306.94,757.99],"aVR":[127.42,128.81,179.38,126.19,318.73,757.97],
 "aVL":[154.50,128.62,182.76,169.37,299.19,758.08],"aVF":[141.00,125.02,184.05,142.60,310.43,758.06],
 "V1":[140.78,129.05,180.72,160.57,303.89,758.06],"V2":[155.93,136.12,176.15,181.65,287.71,757.90],
 "V3":[154.23,132.80,179.01,174.20,285.50,758.01],"V4":[140.60,127.32,180.05,148.55,290.09,758.03],
 "V5":[128.69,123.89,177.39,126.73,310.74,758.12],"V6":[123.44,126.55,174.63,118.63,320.51,758.06]}
T6_ORDER = ["Pdur","QRSdur","Tdur","PQint","QTint","RRint"]
```

```
# Preprint Figure 5 (lead II) approximate peak (mode) read off the figure  [ms / mV]
# NOTE: estimated by eye from the density peaks; the preprint has no raw table.
PRE_FIG5_II = {"Pdur":128,"QRSdur":125,"Tdur":185,"PRint":190,"QTint":385,"RRint":760}


# ---------- load only the columns we need ----------
usecols = ["disease_class","record_id","lead"] + list(COLMAP.values())
df = pd.read_csv(FID, usecols=usecols, na_values=["","None"],
                 dtype={c:"float32" for c in COLMAP.values()})
df = df.rename(columns={v:k for k,v in COLMAP.items()})


# ---------- LEVEL 1: per (class, record, lead) median over beats ----------
lvl1 = df.groupby(["disease_class","record_id","lead"], observed=True)[FEATS].median().reset_index()
lvl1.to_csv(os.path.join(DATA,"per_signal_median.csv"), index=False)
print(f"Level 1: per_signal_median.csv  -> {len(lvl1):,} rows (record x lead)")


# ---------- LEVEL 2: per (class, lead) mean/SD/n across records ----------
agg = lvl1.groupby(["disease_class","lead"], observed=True)[FEATS].agg(["mean","std","count"])
agg.columns = [f"{f}_{s}" for f,s in agg.columns]; agg = agg.reset_index()
agg.to_csv(os.path.join(DATA,"per_lead_class_summary.csv"), index=False)
print(f"Level 2: per_lead_class_summary.csv -> {len(agg)} rows (class x lead)")


# ---------- COMPARISON: sinus per-lead means vs Table 6 (+ Fig5 for lead II) ----------
sin = lvl1[lvl1.disease_class=="sinus"]
rows=[]
for ld in LEADS:
    s = sin[sin.lead==ld]
    row = {"lead":ld}
    for i,f in enumerate(T6_ORDER):
        our = s[f].mean(); row[f"{f}_our"]=round(our,1); row[f"{f}_T6"]=T6[ld][i]
        row[f"{f}_dT6"]=round(our-T6[ld][i],1)
    rows.append(row)
comp = pd.DataFrame(rows)
```

```
comp.to_csv(os.path.join(DATA,"comparison_sinus_vs_table6.csv"), index=False)



print("\n=== sinus per-lead: our mean  vs Table 6  (Δ) ===")

print(f"{'lead':4s}"+ "".join(f"{f:>22s}" for f in T6_ORDER))

for _,r in comp.iterrows():

    print(f"{r['lead']:4s}"+ "".join(f"  {r[f+'_our']:6.1f}/{r[f+'_T6']:6.1f}({r[f+'_dT6']:+5.1f})" for f
in T6_ORDER))



print("\n=== lead II: our mean vs Table 6 vs preprint Fig 5 (estimated) ===")

s2 = sin[sin.lead=="II"]

print(f"{'feature':8s} {'our':>8s} {'Table6':>8s} {'Fig5~':>8s}")

for f in ["Pdur","QRSdur","Tdur","PQint","PRint","QTint","QTpeak","RRint"]:

    t6 = T6["II"][T6_ORDER.index(f)] if f in T6_ORDER else None

    fg = PRE_FIG5_II.get(f)

    print(f"{f:8s} {s2[f].mean():8.1f} {('%.1f'%t6) if t6 else '   -':>8s} {(str(fg)) if fg else '
-':>8s}")

print("\nKEY: QTint our ~%.0f matches preprint Fig 5 (~385), NOT Table 6 (317)." % s2.QTint.mean())
```